Header, conventions, and D2 Overview.

Team: Abby Kraft, Veronica Malusky, Jake Martin, Sophia Munoz

Web-Based Interactive Model Visualizer

Goal: Create a web-based Model visualizer that allows users to quickly change layers of the model as well as create models based on images uploaded via AI.

Overview:

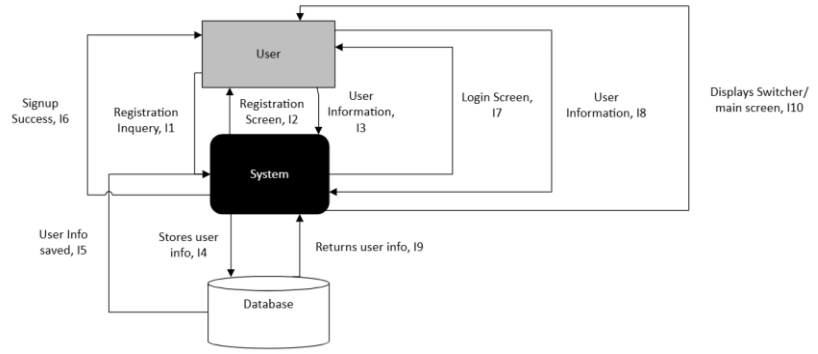
- What the System is: Web based application that allows users to upload images that get converted into 3D models and allow the user to interact and view the data in a volumetric setting, with slicing and analysis from AI.
- What D1 Covered: Image Upload, database and storage solutions.
- What D2 adds: The logic for the login/signup section, the 3D model viewer, and our switcher which is the main screen. D2 also adds edge case handling and rationale for certain design decisions.

Known Gaps and Open Questions:

- Did we want to incorporate AI to count as the image/model generator as well – Allow the user to write a prompt and have AI generate the model.
- What security vulnerabilities could come up
- Use userID, token, or cookie for sessionInfo

Finished components, the D2 diagrams.

- **Image Uploader:**
  - Completed in D1.
- **Login/Signup Section**:
  - Data Model:



- *Conventions: Cylinder = Database, Black rectangle = System, Grey rectangle = User. I(n) where n = 1 – 7 labels the iteration in order from signup to login phases. Each line is labeled with what would get passed to the system, database or user. Most system to user lines are screens or visuals, user to system lines are user information, system to database lines are*

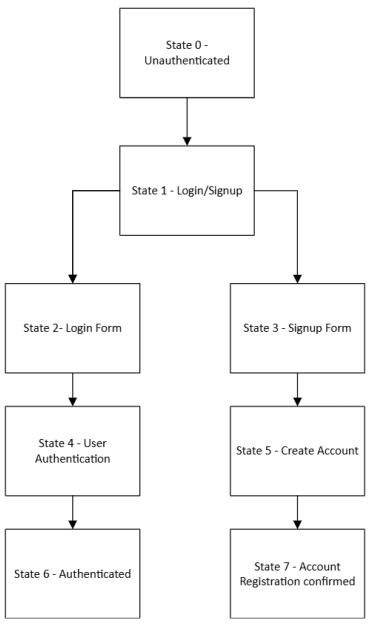
- Goal Statement:
  - Log in or create new user information, grab the user information, log any errors that occur upon log in, or when grabbing the information
- Behavior as an Algorithm Entry:
  - *What it is and Problem it solves:*
    - This will create a user row on a user signup, or when logging in it will verify the credentials, and returning a session token. Solves problem of keeping user privacy. The failures will be written to the error log.
  - *Inputs and Outputs (Format: Name, Type):*
    - *Inputs:* [email, string], [password, string], [firstName, lastName, string], [signUp, Bit]
    - Outputs: [sessionToken, token], user: {userID: int, firstName: string, lastName: string, }], [error, Enum]
  - *Steps:*
    - Validate every field and if invalid will return INVALID_INPUT. Front end checks first, but server repeats check.
    - For a user to sign up, it will either look for the user.email and if it is found there will be a return of EMAIL_ALREADY_TAKEN error. Otherwise, the password will be hashed with a reused library like Argon2id and bcrypt (or java specific Spring Security) and row will be inserted. The signup will be based on a first come first serve basis, so there is no racing.

- For a user to login, it will lookup user.email and compare against the password_hash. If the user enters no email or no password there will be a return of INVALID_CREDENTIALS (this will be a check on front-end to reduce http requests) and if the password is wrong, it will also return INVALID_CREDENTIALS.
- For the log, on failure the error code, component, and time will be logged.

- *Expected Complexity:*
  - The validation is O (1). The email lookup is O(logn) because of the index of user.email. The password hashing will be O (1) in the number of users. The token signing and verification are O (1).
- *Why this Approach Rather than the Alternatives:*
  - This approach of utilizing the hashing library and token mechanism is chosen rather than alternatives because of the security risks of authentication. A third-party login could remove that password storage but adds an external dependency and requires every user to have an outside account.

- State Machine or Step List:

States:



- Specific States may require reentry of information, such as state 4 and state 7, in the event the user inputs the wrong user information in

state 4 they will be prompted to reenter their information and if the user completes their account registration they will need to reinput their credentials to actually sign in. Once authenticated, the system will display the switcher screen (main screen).

- Interface as endpoints, message formats, or signatures with errors:
  - POST auth
    Login example
    JSON
    {
    Email: “example@mail.uc.edu”,
    Password: “Example”,
    FirstName: “”,
    LastName: “”,
    Signup: 0
    }
    Signup example
    JSON
    {
    Email: “example@mail.uc.edu”,
    Password: “Example”,
    FirstName: “Test”,
    LastName: “Test”,
    Signup: 1
    }

  Endpoint auth would be a single endpoint for both login and signup. With the Signup bit it would determine in the function whether or not the code needs to go through the sign up information or the login. Each email needs to be unique as mentioned above. This is where the error messages would return in the response message.

  201 – Created – Account created (would take the user to the login screen to re-enter their information to sign in)

  200 – OK – Login Successful (would return user information)

  400 – Invalid credentials – error to prompt user to retry sign in (incorrect user or password)

401 – Identical User – error when attempting to create account (email already used)

200 – OK response example:

{

SessionToken: token

User:

{

userID: 1,

FirstName: “Example”,

LastName: “Test”,

GLBModels:

{

Model info from D0:

“ID”: Primary Key,

“Title”: User Inputted Name,

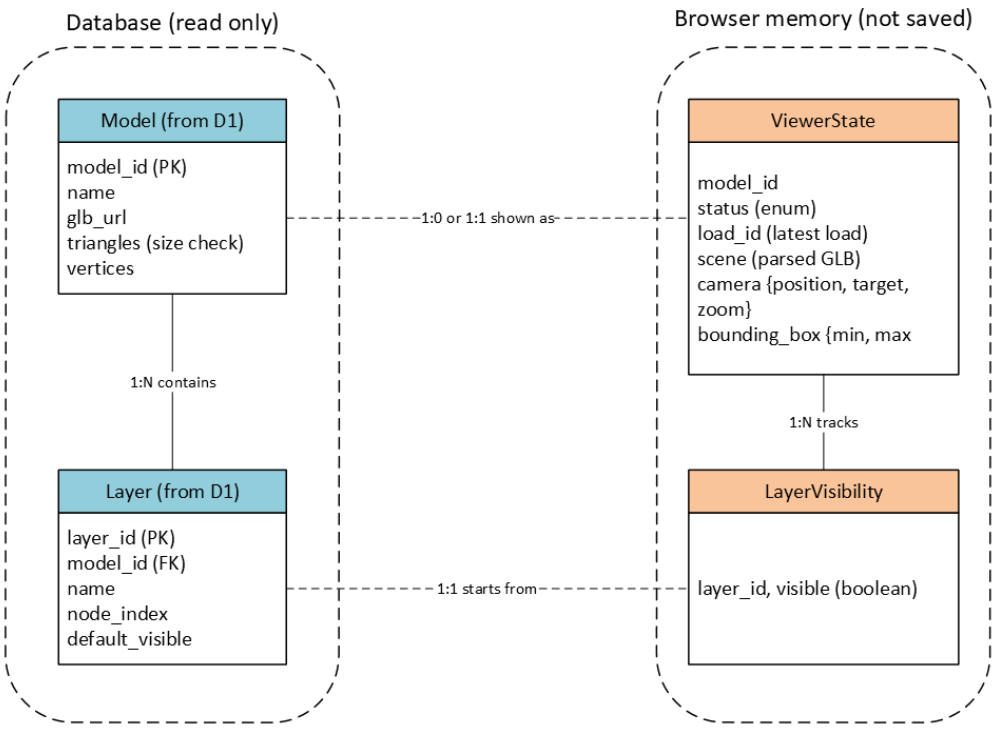
“FileURL”: GLB URL

}

}

}

- **3D Model Viewer:**
  - Data Model:



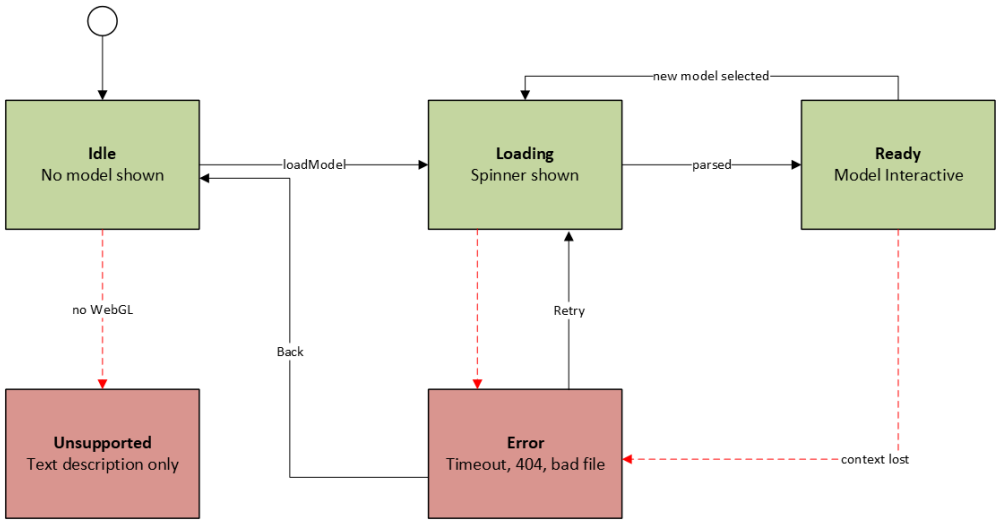
- There isn't any persistent data for the viewer. It never actually writes to the database. When the model is selected, it reads the model's row from Model and its row from Layer and copies the necessary information into the browser memory. ViewerState holds the parsed model, load status, and camera. And each LayerVisibility starts from its layer's default_visible and then changes when the user selects a layer. When the user picks another model or closes the tab, all of this information is discarded.
- *Conventions:*
  - Blue box: stored table the viewer reads (never writes)
  - Orange box: object held in the browser memory only
  - Dashed box: where the data lives
  - Solid line: relationship
  - Label: cardinality and name
  - Dashed line: copied into memory on load, never written back
  - PK: primary key
  - FK: foreign key

- Goal Statement:
  - *Render selected model in user's browser so they can rotate, zoom, and switch layers, and fail visibly with an error message.*
- Behavior as an Algorithm Entry:

- *What it is and Problem it solves:*
    - The 3D model viewer will render the selected 3D model GLB in the browser using WebGL, also taking the visibility from the switcher, on the user's device, so the user can inspect and interact with the model. This solves the problem of allowing users to inspect a model smoothly and of failing visibly rather than acting like it loaded and showing a blank canvas.
- *Inputs and Outputs (Format: Name, Type):*
    - *Inputs:* [glbUrl, string (/api/v1/models/model_[1D])], [layerState, map(layerId(int) -> visible (boolean)], [camera input, pointer, touch, keyboard events].
    - *Outputs:* [rendered scene, WebGl canvas], [error, Enum]
- *Steps:*
    - Make sure that WebGL is available, if it is not then UNSUPPORTED_BROWSER with an on-screen message will be returned.
    - If a model was already on the canvas, then discard the previous model materials, so memory doesn't build up.
    - When loading, there will be a loading indicator and request the GLB with a timeout after so long. If a newer load request comes up, then it will cancel and discard any that is in the process of loading.
    - Parse with loader library. If a network or parse error comes up, there will be a LOAD_FAILED popup on screen.
    - Model gets added to the scene, fits its bounding box, then apply layerState.
    - Possible render loop if there is a camera input needed.
    - On layer change, update only that layer's visibility without having to reload the entire file.
- *Expected Complexity:*
    - The certain loading and parsing are of O (g) where g is thesize of the GLB. The fitting of the camera is O (1) from a stored bounding box and O (v) over vertices. Applying full layer state is O (1) for 1 later and the toggle would be O (1). The frame costs O (t) depending on triangles drawn on users' GPU.
- *Why this Approach Rather than the Alternatives:*

- This approach was chosen rather than an alternative because of our team's experience with front-end development. Also, the server-side rendering was rejected because of the GPU cost. The visibility was chosen to save cost and memory.

- State Machine or Step List:



  - Conventions:
    - Box: state
    - green: normal state
    - red: error state
    - dot: start
    - solid arrow: normal transition
    - red dashed arrow: error path
    - arrow label: event causing transition

- Interface as endpoints, message formats, or signatures with errors:
  - The interface as signatures with errors. The methods are called by the switcher, the viewer has no HTTP endpoints of its own
    - Method: loadModel
      - Inputs:
        - modelId: int, required >= 1
        - glbUrl: string, required /api/v1/models/model_[ID]
        - LayerState: map<int, bool>, optional (default, every layer's default_visible)

        - Output: Promis<LoadResult>: {modelId: int, vertices: int, triangles: int, layerIds: int[], loadMs: int (milliseconds)}
        - Errors: UNSUPPORTED_BROWSER (no WebGL), LOAD_TIMEOUT (over 15s), LOAD_FAILED (network error, 403/404), PARSE_FAILED (corrupt GLB), MODEL_TOO_LARGE (over 50 MB or 2,000,000 triangles), CANCELLED (newer load replaced it, switcher ignores it)
    - Method: setLayerVisbility
        - Inputs:
            - layerId: int, required
            - visible: boolean, required
        - Output: void
        - Errors: NO_MODEL_LOADED (called before Ready) - unknow layerId is ignored and logged, not thrown
    - Method: resetCamera
        - Inputs: none
        - Output: void
        - Errors: NO_MODEL_LOADED
    - Method: getDescription
        - Inputs: none
        - Output:
            - String: model name, visible layer names, vertex count
        - Errors: none, works even UNSUPPORTED
    - Method: dispose
        - Inputs: none
        - Output: void – frees up memory in the GPU
        - Errors: none
- Errors reach the switcher as {code: enum, message: string, recoverable: boolean}
    - Examples:
      // switcher to viewer request
      await viewer.loadModel(42, "/api/v1/models/model_42", new Map([[7, true], [8, false]]));
      // success

{ "modelId": 42, "vertices": 45218, "triangles": 87342, "layerIds": [7, 8], "loadMs": 1840 }
// failure
{"code": "LOAD_TIMEOUT", "message": "the model took too long to load", "recoverable": true }

- **Switcher:**
  - Goal Statement:
    - *The goal of the switcher is to allow quick and easy switching of the model as well as the layers of the 3d models*.
  - Behavior as an Algorithm Entry:
    - *What it is and Problem it solves:*
      - This will show the user their models since they are signed in and tracks which layers are visible and sends that selection to the viewer. Also, receives new models from the image uploader. This solves the problem of reaching any saved model quickly without having to reupload the image.
    - *Inputs and Outputs (Format: Name, Type):*
      - *Inputs:* [sessionToken, string], [modelId, int], [layer toggle, {layerId: int, visible: boolean}], [new model, modelId from image upload].
      - *Outputs:* [error, Enum], [modelList, array {modelId, name, format, vertices, trianges}], [selection, {modelId, glbUrl, layers: [{layerId, name, visible}]}].
    - *Steps:*
      - Screen load and first verify the token, if invalid then redirect to login.
      - Request for the first page of the user's models, done using the model.user_id index. If the result is empty, show an empty result with a prompt start using it.
      - When a model is picked by the user, it confirms that it is theirs, and then a fetch is to its layer rows using the layer.model_id index.
      - LayerState is built based on each layer's default visibility and then is sent as selection to Viewer.
      - When there is a layer toggle, the layerState is updated and sends the change to the viewer.

        - The models will be in order of earliest / newest at the top of the list. If the image uploader reports failure, a popup will show on screen saying, “the image wasn’t able to be rendered into a 3D model”.
        - With a failure of a list or fetch, there will be an error shown on screen with a retry.
    - *Expected Complexity:*
        - Fetching a page will be done using the model.user_id index with M being stored models and L is the page size so it is O (log M+L). Building the layerState is O (1) with a toggle being O (1), and rendering the list is O (L). Per screen cost does not grow with M. Fetching layers is done with layer.Model_id with LY being total layer rows with there being 1 starting layer in the model so it is O (log LY +1).
    - *Why this Approach Rather than the Alternatives:*
        - The paging with a limit will mitigate client storage and request bottlenecks, versus loading the whole library at once. Listing the data separately from the GLB file means the list will appear immediately, and the file loads only when chosen.

Edge cases and error handling

ID	Component	Category	The Edge Case	Design Decision	Decided Behavior	Handler	Recoverable
EC-U-01	Image Uploader	Boundary values	File is exactly 20 MB, or 1 byte over; 0-byte file	<= 20 MB is accepted. Over 20 MB -> 413. 0 bytes -> 400. Checked in the browser first, repeated on the server	Surface error	Upload.beforeUpload (front-end) UploadController.validate (server)	Yes, user can pick another file
EC-U-02	Image Uploader	Boundary values	Resolution outside	Over 4096 -> downscaled to 4096 before	Degrade/surface error	UploadController.validate	Yes

			256-4096px	sending to AI. Under 256 -> 400 message “image too small”			
EC-U-03	Image Uploader	Timing and concurrency	AI Service doesn't respond within 60s	Job marked FAILED with error_code = AI_TIMEOUT; one automatic retry, then 422 to user	Retry, then surface error	GenerationJob worker	Yes, user can still resubmit
EC-U-04	Image Uploader	Timing and concurrency	User closes the tab mid-generation	Job continues on the server; model appears in the switch on the next login	Fall back	GenerationJob worker	Yes, nothing is lost
EC-U-05	Image Uploader	Timing and concurrency	User clicks Upload twice/re-uploads the same image	Button disabled while a job runs; matching Upload.image_hash returns the existing model instead of a new AI call	Fall back	Upload.beforeUpload, UploadController hash lookup	Yes
EC-U-06	Image Uploader	Bad or missing input	File renamed to .png but isn’t an image	Header bytes checked, not the extension -> 415	Surface error	UploadController.validate	Yes
EC-U-07	Image Uploader	Bad or missing input	Image has no recognizable object; AI returns an empty or invalid GLB	GLB validation (D1 algo. 2) rejects it -> 422 message “couldn’t build a model from this image”	Surface error	GlbValidator	Yes, user uploads a clearer image
EC-U-08	Image Uploader	External dependencies	AI service rate limit or quota hit (429)	Retry up to 3 times with 2s/4s/8s waits, then message "service busy, try later"	Retry, then surface error	AiClient	Yes
EC-U-09	Image Uploader	External dependencies	File storage write fails after a success	Retry up to 3 times, if it still fails, job FAILED and error logged. The AI result is	Retry, then surface error	ModelStore.save	Yes

			ful generation	not charged again on resubmit if hash matches a stored GLB			
EC-L-01	Login/Signup Section	Boundary values	Password under 9 or over 72 characters, email over 243 characters	Rejected with INVALID_INPUT. 72 is bcrypt limit, longer passwords would be silently cut off	Surface error	Signup form, AuthService.validate	Yes
EC-L-02	Login/Signup Section	Timing and concurrency	Two signups with the same email at the same moment	Unique index on User.email makes the second insert fail -> EMAIL_ALREADY_TAKEN (enforces FCFS)	Surface error	AuthService.signup	Yes
EC-L-03	Login/Singup Section	Timing and Concurrency	Session token expires while the user is in the app	Next request gets 401 -> redirect to login, then back to the switcher	Fall back	AuthMiddleware, front-end API wrapper	Yes
EC-L-04	Login/Signup Section	Bad or missing input	Wrong password repeated (potential guessing attack)	After 5 failures in 15 minutes, account is locked 15 min -> TOO_MANY_ATTEMPTS. Same INVALID_CREDENTIALS message shown whether the email exists or not	Surface error	AuthService.login	Yes, after lockout ends
EC-L-05	Login/Signup Section	Bad or missing input	Malformed email or input contains SQL	Format check -> INVALID_INPUT; all the queries are parameterized, so input is never run as SQL	Surface error	AuthService.validate, database layer	Yes
EC-L-06	Login/Signup Section	External dependencies	Database unreachable	503 message "service unavailable, try again", error is then logged	Surface error	AuthService, ErrorLog	Yes, once the DB is back

				with component and time			
EC-V-01	3D Model Viewer	Boundary values	GLB over 50 MB or 2,000,000 triangles	Checked against Model.triangles before downloading -> MODEL_TOO_LARGE. Between 500,000 and 2,000,000 triangles: load, but warn it may be slow on phones	Surface error / degrade	Viewer.loadModel	Not for that model, but yes for others
EC-V-02	3D Model Viewer	Boundary values	Model is flat or a single point (bounding box is size 0)	Camera placed at a default distance instead of dividing by 0	Fall back	Viewer.fitCamera	No
EC-V-03	3D Model Viewer	Timing and concurrency	Download takes longer than 15s	LOAD_TIMEOUT with a retry button	Surface error	Viewer.loadModel	Yes
EC-V-04	3D Model Viewer	Timing and concurrency	New model selected before the previous finished	Older download aborted, any result with an old loadId is discarded so the wrong model never appears	Fall back	Viewer.loadModel	Yes
EC-V-05	3D Model Viewer	Timing and concurrency	Tab hidden while a model is shown	Render loop paused to save battery and GPU, and then resumes when visible	Degrade	Viewer visibility listener	Yes
EC-V-06	3D Model Viewer	Bad or missing input	GLB corrupt or glbUrl returns 404	PARSE_FAILED or LOAD_FAILED message. The old GPU memory is freed, user returned to Error state with Retry and Back	Surface error	Viewer.loadModel	Retry for network errors, no for a corrupt file
EC-V-07	3D Model Viewer	Bad or missing input	layerState names a layer	The layer is ignored and a warning is logged, the rest still applies	Degrade	Viewer.setLayerVisibility	Yes

			not in the GLB				
EC-V-08	3D Model Viewer	External dependencies	Browser has no WebGL	UNSUPPORTED_BROWSER, shows the test description from getDescription() instead of a canvas	Fall back	Viewer startup check	No 3D in the browser, text would still work
EC-V-09	3D Model Viewer	External dependencies	WebGL context lost (GPU reset or out of memory)	Reload the model automatically once, if it's lost again, CONTEXT_LOST error	Retry, then surface error	Viewer context-lost listener	Yes
EC-S-01	Switcher (Main Screen)	Boundary values	User has zero models	Empty state with a prompt to upload an image	Fall back	Switcher.fetchModels	No
EC-S-02	Switcher (Main Screen)	Boundary values	Model has no Layer rows	Treated as one layer called "Whole model", is always visible	degrade	Switcher.buildLayerState	No
EC-S-03	Switcher (Main Screen)	Timing and concurrency	Upload finishes while the user is browsing the list	New model is inserted at the top without reloading the list	Fall back	Switcher.onNewModel	No
EC-S-04	Switcher (Main Screen)	Timing and concurrency	User picks model B while model A is still loading	Latest selection wins, A's load is cancelled (EC-V-04)	Fall back	Switcher.select, Viewer.loadModel	Yes
EC-S-05	Switcher (Main Screen)	Bad or missing input	modelId belongs to another user (edited URL)	404, same as a missing model, so other users' model IDs aren't revealed	Surface error	ModelController.get ownership check	Yes
EC-S-06	Switcher (Main Screen)	External dependencies	Model list or layer fetch fails	Error banner with a Retry button, with one automatic retry first	Retry, then surface error	Switcher.fetchModels	Yes

Requirements traceability table.

Component table:

Requirement (functional and non-functional)	ID and Short Text	Priority	Design Elements that Satisfy it (DELETE BEFORE TURNIN (an entity, endpoint, algorithm, or figure, not a component name alone),	How the Design Elements Satisfy it
username	username	top	This will be implemented as a text box	Allows user to input username
password	password	top	This will be implemented as a text box	Allows user to input
Sign in button	Sign-in	top	This will be implemented as a button connected to the back end	Allows users to Sign-in
User_3d_ Models	3d-mods	top	This will be an array of 3d models returned from the sign in process	is the users 3d models
User cookie	cookie	low	This will be a site cookie returned from the sign-in process	Holds the hashed user data allowing user to stay logged into the site
Sign-in	Sign-in process	top	Sends the user data either from the cookie or from the text boxes to the backend where the sign in process then confirms the username and password	Is the process needed to sign in
Click and drag model	Click_drag	med	3d viewer implements click and drag capabilities	It allows user to see the 3d model from multiple angles
Mouse scroll	scroll	med	3d vier implements the zoom in and out when user scrolls on the scroll wheel	Allows user to see greater detail on the model

View model	model	top	This will be implemented through a prebuilt 3d model viewer	Allows user to view model
Model list	Model_arr	top	This will be accomplished in the model switcher with a list of clickable buttons each for a different model	This will allow user to quickly see available models
Model List item	Model_item	top	This will be a button linked to the backend with the model information allowing users to see the model in the viewer when clicked	This will allow users to quickly change which model they are viewing
Model-layer switch	Model-switch	med	A radio button can be used to complete this where it is always either showing	Allows the user to either see the available models or the layers of the model they are on
Model level	Model_level	med	A set of buttons with the different layers of the 3d model, allows users to quickly change what level they are looking at.	Allows user to quickly see different layers of the 3d model.

Traceability table:

event	link	priority	notes
login	Username-> password->sign in	med	This is the process to login
View model	Model switcher->model-layer switcher-> model-list->model-item ->model-viewer	med	This allows a user to load a new model
Change layer	Model switcher->model-layer switcher-> model-layer-> model-viewer	high	Allows a user to quickly change the user

Generate model	Model generator->model-switcher->model viewer	high	Sending the model from the generator to the viewer

Reverse Traceability table

event	Elements needed	Scope conclusion
login	Username, password, sign in	This is a must for the final project but early version can be rather crude
View model	Model switcher, model-layer switcher, model-list, model-item , model-viewer	This will need to be implemented early on does not need to implement tons of functionality at first
Change layer	Model switcher, model-layer switcher, model-layer, model-viewer	This is tyhe point of the project and the innovation we hope to bring, it is rather important to the project
Generate model	Model generator, model-switcher, model viewer	This ia major need of the project and needs to be done to the best of our ability

Design rationale.

Significant D1 and D2 decisions:

1. We chose a relational database (PostgreSQL) over a document store because our data has clear relationships that are queried together, and because job status needs consistency. We accepted the trade-off of a fixed schema that must be migrated when it changes because of our stable entities and the importance of enforced relationships.
2. We chose passing a GLB URL over passing the GLB file because the switcher should probably not hold model binaries and the viewer mist be able to cancel a stale download. We accepted the trade-off of one extra network request per model because the download is cached by the browser.
3. We chose to use SessionTokens as a part of the Login phase to improve security and user experience by reducing the amount of times they are required to login. The

backend would verify the session before the user can upload a new model or open a model through the switcher. This was done over than returning the userID and using that to obtain everything for security purposes.

4. We chose to limit the models that display in the switcher to what fits on screen size (5 models for example) then allow for additional API calls on user scroll. This is to reduce the amount of models we would be sending to the front end per call. Reducing the response size, which would improve reaction time of the server. This change was added from peer review, originally we were going to send all the models, but this would not handle well as number of models increased.
5. A choice was made to make the layer and model viewer into one component. This will allow for a nice clean form factor. It also gives the viewer on visual place to look when changing around the model. This design will also help when it comes to the loading of the layers, as the switcher can load it without having to show an extra loader for the layers as it will not be visible to the user. This does come at the trade off of it being a more complicated component but it will be well worth it.
6. The decision to save the user's models when they are generated certainly has drawn backs when it comes to storage, but it seems to be a good step towards the end goal. If a new model had to be generated each time, the cost in Ai would be much greater and it would leave the user with a worse experience.

]

Peer design review notes.

Reviewed team: Abby Kraft, Veronica Malusky, Jake Martin, Sophia Munoz
Reviewing team: Kyle Russell, Shashank Sathiyanarayanan, Jaxon Polo, Arleen Monteiro, Nyla Spencer

What we liked

Your algorithm entries were easy to follow, and we liked that input gets checked on both the front end and the server. The Viewer canceling stale loads and freeing old materials is a smart touch, and loading the model list separately from the GLB file makes sense for speed.

Feedback

Feasibility

- Your overview talks about volumetric slicing, but a GLB file is just a surface mesh. We weren't sure where the layers come from. How does one uploaded image turn into layers?
- The AI image-to-3D step seems like the heart of the project, but the doc never says which model or service you'll use. Is that something your team can realistically get working this semester?
- The signup logic says there's no racing, but two people could sign up with the same email at the same moment and both get through the check. A unique constraint on the email column would fix that.

Clarity

- Field names and types shift between sections. For example, email and Email both show up, and sessionToken is a "token" in Login but a "string" in the Switcher.
- Using one /auth endpoint with a Signup bit made it harder for us to tell what errors each case returns. Two separate endpoints might be simpler.
- The complexity notes were a little tough to read. Listing each step with its cost would help.

Risk

- If the AI service fails, you retry once and then give up. What happens when it returns a broken mesh? Keeping the uploaded image so the user can try again would be a good safety net.
- We couldn't tell if the model URL checks that the person asking actually owns the model.
- Token expiry, where the token is stored, and logout aren't defined yet, and the doc lists Argon2id and bcrypt without picking one.
- There's no size limit on GLB files, which could cause trouble on lower-end devices.

Action items

ID	Criterion	Issue	Action	Owner	Priority
PR-01	Feasibility	Not clear where layers come from	Explain how layers are made from one image, or tone down the volumetric claim		Must-fix
PR-02	Feasibility	AI approach not stated	Name the model or service and a backup plan for poor results		Must-fix

PR-03	Feasibility	Two signups can collide	Add a unique constraint on user.email		Must-fix
PR-04	Risk	No fallback if the AI fails	Keep the uploaded image for retry and check the returned mesh		Must-fix
PR-05	Risk	Model URL ownership unclear	Check ownership on the model endpoint and say how the token is sent		Must-fix
PR-06	Risk	Auth details missing	Choose one hashing method, set token expiry, decide storage and logout		Must-fix
PR-07	Risk	No GLB size limit	Set a max file size or triangle count		Worth considering
PR-08	Clarity	Names and types don't match	Make them consistent across Login, Switcher, and Viewer		Must-fix
PR-09	Clarity	/auth does too much	Split into separate signup and login endpoints		Worth considering
PR-10	Feasibility	Prompt-based generation idea	Save for future work		Out of scope for now

Summary

Most of our feedback fell under Risk, mainly around the AI service and login. Our two biggest concerns were:

1. How the image-to-3D step actually produces the layers the Viewer needs.
2. The missing failure and security decisions for the AI service and the login flow.

Summary

Reviewed team: Kyle Russell, Shashank Sathiyanarayanan, Jaxon Polo, Arleen Monteiro, Nyla Spencer
Reviewing team: Abby Kraft, Veronica Malusky, Jake Martin, Sophia Munoz

The main 2 criterion of concern ended up being clarity and feasibility. Some parts of the document were difficult to read, due to layout and model issues.

Our two biggest concerns from reviewing the team:

1. Data models had some issues with overlapping and unreadable in some parts and not yet being demonstrated

2. Missing approver information regarding gate threshold and only having one approver for the given information